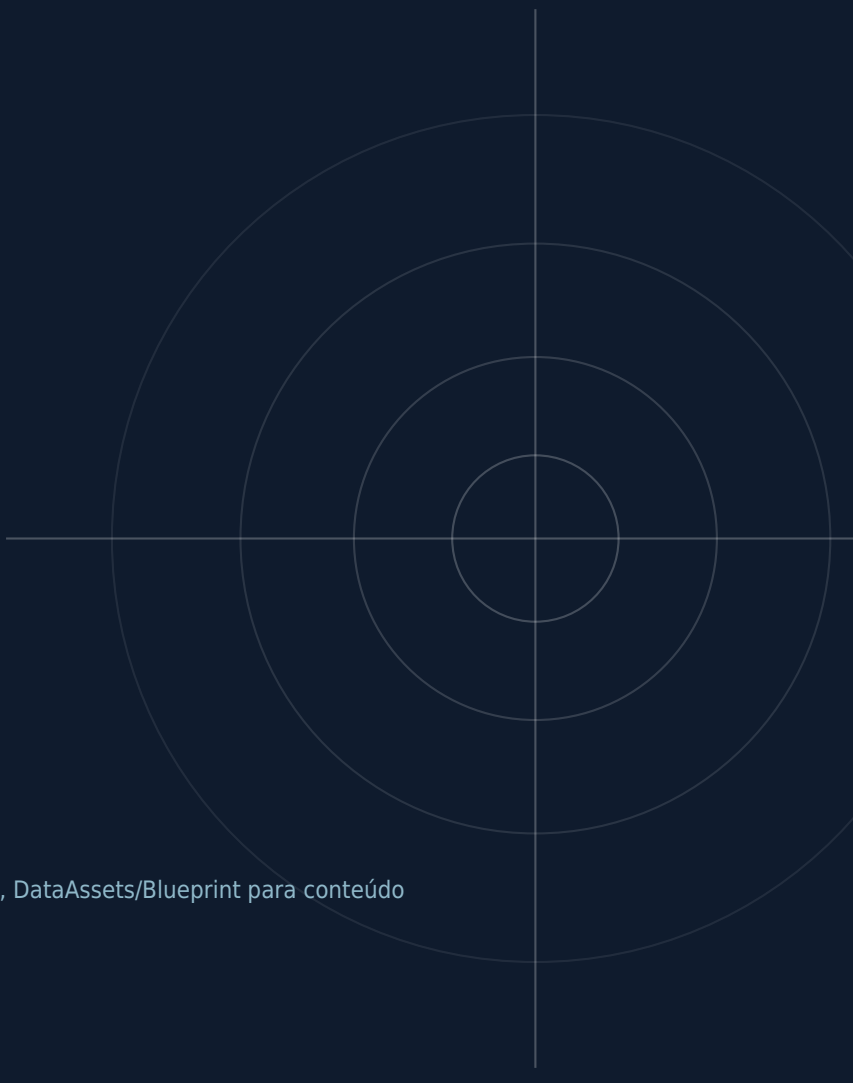
DOCUMENTAÇÃO TÉCNICA

IA Tática para Bots

FPS tático em Unreal Engine 5

Arquitetura de referência: percepção imperfeita, decisão em layers, raciocínio tático, coordenação de esquadrão e humanização.



Versão 1.0 · 29 de outubro de 2026

Alvo: Unreal Engine 5.x · C++ para o núcleo, DataAssets/Blueprint para conteúdo

Sumário

1. Visão geral . . . 4
1.1 Objetivo . . . 4
1.2 Escopo . . . 4
1.3 Princípios de design . . . 4
1.4 Mapeamento: Tarkov/SPT para Unreal . . . 4
1.5 Premissas e avisos . . . 5

2. Arquitetura geral . . . 5
2.1 Módulos e classes . . . 5
2.2 Fluxo de dados por frame . . . 6
2.3 Estrutura de pastas do plugin . . . 6

3. Percepção . . . 6
3.1 Visão . . . 6
3.2 Acumulador de consciência . . . 7
3.3 Audição . . . 7
3.4 Dano e equipe . . . 8
3.5 Interface com as crenças . . . 8

4. Modelo de conhecimento (crenças) . . . 8
4.1 Estruturas . . . 8
4.2 Regras de atualização . . . 9
4.3 Memória de longo prazo . . . 9
4.4 Regras anti-wallhack . . . 9

5. Decisão: brains e layers . . . 9
5.1 Conceito . . . 9
5.2 Contrato de uma layer . . . 9
5.3 Arbiter . . . 10
5.4 Catálogo de layers . . . 10
5.5 Dentro de uma layer . . . 10
5.6 Extensibilidade (o equivalente ao BigBrain) . . . 11
5.7 Estabilidade (anti-flicker) . . . 11

6. Raciocínio tático . . . 12
6.1 Dados de cobertura . . . 12
6.2 Pontuação de posições . . . 12
6.3 Reservas . . . 12
6.4 Catálogo de queries EQS . . . 13
6.5 Movimento sob fogo . . . 13
6.6 Granadas . . . 13

7. Ciclo de combate . . . 13
7.1 Moral (lutar ou fugir) . . . 14

7.2 Supressão . . . 14

8. Esquadrão . . . 14

8.1 Estrutura . . . 14
8.2 Conhecimento compartilhado . . . 14
8.3 Planner do esquadrão . . . 15
8.4 Papéis . . . 15
8.5 Tokens . . . 15
8.6 Comunicação (barks) . . . 16
8.7 Espaçamento e fogo amigo . . . 16

9. Execução e humanização . . . 16

9.1 Movimento . . . 16
9.2 Mira falível . . . 16
9.3 Animação . . . 17
9.4 Justiça e legibilidade . . . 17
9.5 Imperfeições deliberadas . . . 17

10. Perfis, personalidades e dificuldade . . . 17

10.1 UBotProfile . . . 17
10.2 Faixas de dificuldade (ponto de partida) . . . 18
10.3 Personalidades . . . 18
10.4 Dificuldade dinâmica (opcional) . . . 18

11. Multiplayer e rede . . . 18

12. Performance . . . 19

12.1 Orçamentos . . . 19
12.2 AI LOD . . . 19
12.3 Técnicas . . . 19

13. Debug e ferramentas . . . 19

14. Testes e qualidade . . . 20

14.1 Camadas de teste . . . 20
14.2 Cenários de arena . . . 20
14.3 Métricas . . . 20

15. Roadmap de implementação . . . 20

16. Riscos e armadilhas comuns . . . 21

17. Apêndices . . . 21

A. Checklist de implementação / code review . . . 21
B. Glossário . . . 22
C. Referências . . . 22

1. Visão geral

1.1 Objetivo

Este documento define uma **arquitetura de referência** para a IA de bots de um FPS tático na Unreal Engine 5. O alvo é um comportamento que o jogador percebe como "inteligente" sem que o bot trapaceie: ele erra, duvida, perde o jogador de vista, coordena com o esquadrão e reage de forma legível.

A arquitetura combina ideias de sistemas conhecidos: **brains compostos por layers com prioridade** (o modelo que o BigBrain expõe para mods do SPT, usado por SAIN, Questing Bots e similares), **GOAP/planejamento** do F.E.A.R., **Behavior Trees** do Halo e o uso de **tokens e posições táticas** do Killzone.

1.2 Escopo

Dentro do escopo	Fora do escopo
Percepção (visão, audição, dano), modelo de crenças, decisão em layers, cobertura/EQS, combate, esquadrão, humanização, dificuldade, rede, performance, debug e testes.	Aprendizado de máquina/RL, IA de veículos, diálogo narrativo, ferramentas de level design além do bake de cobertura, balanceamento de armas.

1.3 Princípios de design

Princípio	Consequência prática
Mesmas regras, informação limitada	O bot não lê a posição real do jogador. Tudo passa por percepção e crenças. Isso evita a sensação de wallhack.
Decidir sobre crenças, não sobre o mundo	Layers só consultam o modelo de crenças e o contexto do bot. Proibir acessos diretos ao pawn do jogador na camada de decisão.
Dificuldade = imperfeição calibrada	Reação, erro de mira, alcance e taxa de percepção variam por perfil. Nada de dano oculto ou visão através de paredes.
Data-driven e extensível	Brains, perfis e pesos táticos são DataAssets. Novos comportamentos entram como layers, sem editar o núcleo.
Legibilidade para o jogador	Barks, lanternas, lasers e pausas de reação anunciam a intenção do bot. O jogador deve conseguir ler e contra-jogar.
Observável desde o dia 1	Cada decisão tem motivo registrado. Sem ferramentas de debug, IA tática vira adivinhação.
Orçamentos explícitos	Custo por bot e por frame é medido e limitado desde as primeiras fases (seção 12).

1.4 Mapeamento: Tarkov/SPT para Unreal

Para quem conhece o ecossistema SPT, a tabela abaixo traduz os conceitos para esta arquitetura.

Conceito (SPT / BigBrain / SAIN)	Equivalente neste sistema
Brain (um por tipo de bot)	`UBotBrainDefinition` (DataAsset com a lista de layers) identificado por `FGameplayTag`.
Layer (decide quando agir, com prioridade)	`UBotLayer` com `Evaluate()` e `Priority`.
Logic / Action (o que o bot faz)	Tick da layer, que delega para StateTree/BT ou escreve **intents**.
API do BigBrain para mods injetarem layers	`UBotBrainRegistry::AddLayer/RemoveLayer`, chamado por Game Feature Plugins.
Personalidades e dificuldade do SAIN	`UBotProfile` + `UDifficultyTier` (DataAssets com curvas e parâmetros).
Zonas de bot / pontos do mapa	`FCoverPoint`, pontos de patrulha e regiões táticas (seção 6).

1.5 Premissas e avisos

- Versão alvo: **UE 5.4 ou superior**. Nomes de classes e assinaturas de API citados aqui devem ser conferidos na versão em uso, pois o módulo de IA evolui a cada release.
- O núcleo (percepção, brain, esquadrão, combate) é em **C++**. Conteúdo (perfis, layers simples, EQS, animação) fica em DataAssets e Blueprint.
- Os **valores numéricos** são pontos de partida para ajuste (tuning), não verdades. Calibre com playtests.
- Jogo **servidor-autoritativo**: a IA roda no servidor (ou no host/standalone em single player).

2. Arquitetura geral

O sistema é um pipeline de cinco estágios, atravessado por um coordenador de esquadrão. Cada estágio tem responsabilidade única e conversa com o próximo por estruturas de dados simples.

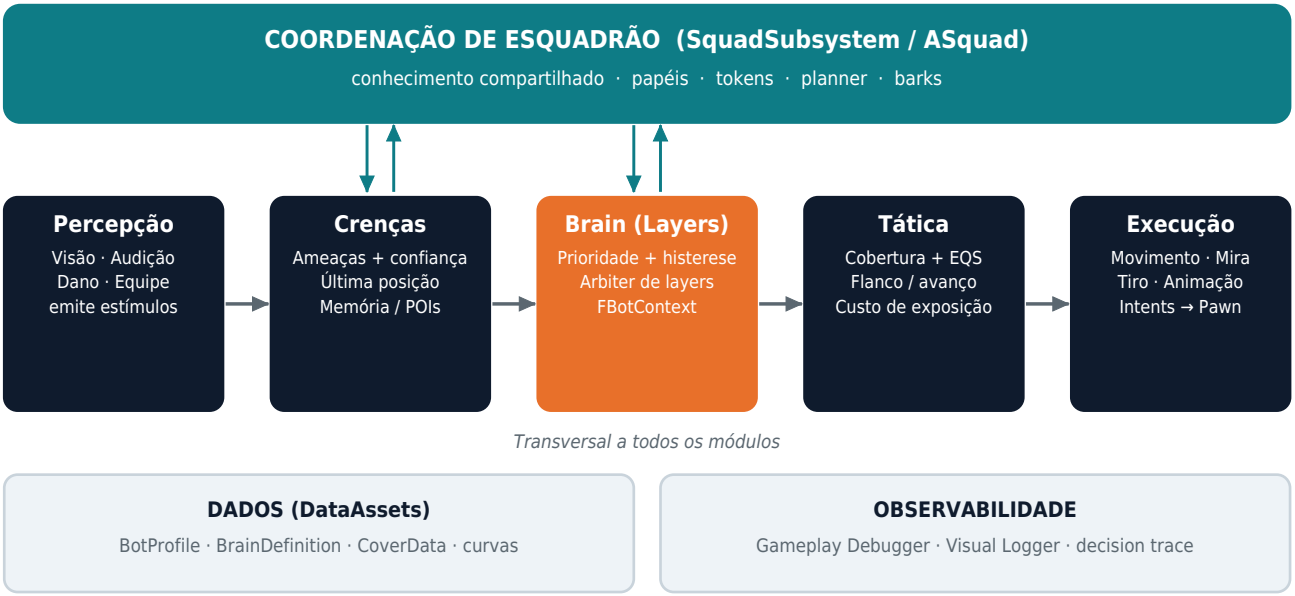


Figura 1. Pipeline de IA e módulos transversais.

2.1 Módulos e classes

Módulo	Responsabilidade	Classes principais
Controller	Dono dos componentes; ponte com o Pawn. Implementa `IGenericTeamAgentInterface`.	`ATacticalAIController`
Percepção	Converte sensores em estímulos normalizados. Wrapper sobre `UAIPerceptionComponent` com modelos próprios.	`UBotPerceptionComponent`, `FSightEvaluator`, `FHearingModel`
Crenças	Mantém ameaças, última posição conhecida, confiança e memória de curto/longo prazo.	`UBotBeliefComponent`, `FThreatBelief`
Brain	Arbitra layers por prioridade, com histerese e registro de decisões.	`UBotBrainComponent`, `UBotLayer`, `UBotBrainRegistry`
Tática	Cobertura, EQS, custo de exposição, seleção de posições.	`UCoverSubsystem`, EQS contexts/tests, `UTacticalNavFilter`
Combate	Mira falível, tiro, recarga, granadas, estados de combate.	`UBotCombatComponent`, `UAimModel`
Esquadrão	Conhecimento compartilhado, papéis, tokens, barks e planner.	`USquadSubsystem`, `ASquad`, `FTokenPool`

Módulo	Responsabilidade	Classes principais
Dados	Perfis, brains, dificuldade, curvas de ajuste.	`UBotProfile`, `UBotBrainDefinition`, `UDifficultyTier`
Debug	Visualização e trace de decisões.	`FGameplayDebuggerCategory_Bot`, `FDecisionTrace`

2.2 Fluxo de dados por frame

- **Orientado a eventos:** sensores empurram estímulos para as crenças quando algo muda (detecção, perda, som).
- **Decisão em intervalo:** o brain avalia layers a cada 0.1 a 0.3 s por bot, com jitter aleatório para não sincronizar bots.
- **Layer ativa:** roda a cada frame ou em intervalo próprio, mas **não** move o pawn diretamente. Ela escreve **intents** (`FBotIntent`: mover para, olhar para, postura, modo de fogo).
- **Executores:** movimento, mira e combate consomem as intents em taxa de frame. Isso separa "o que quero" de "como executo" e facilita trocar executores.

2.3 Estrutura de pastas do plugin

```
Plugins/TacticalAI/Source/TacticalAI/
  Core/        TacticalAIController, BotContext, BotIntent, BotTags
  Perception/  BotPerceptionComponent, SightEvaluator, HearingModel
  Beliefs/     BotBeliefComponent, ThreatBelief, SearchArea
  Brain/       BotBrainComponent, BotLayer, BotBrainDefinition, BotBrainRegistry
  Layers/      Layer_Survive, Layer_Engage, Layer_Investigate, Layer_Patrol ...
  Tactics/     CoverSubsystem, CoverPoint, EQS (contexts/tests), TacticalNavFilter
  Squad/       SquadSubsystem, Squad, SquadPlanner, TokenPool, BarkSystem
  Combat/      BotCombatComponent, AimModel, WeaponHandling
  Data/        BotProfile, DifficultyTier, TuningCurves
  Debug/       GameplayDebuggerCategory_Bot, DecisionTrace
```

Por que um plugin?
Isolar a IA em um plugin (idealmente com Game Feature Plugins para conteúdo) força fronteiras limpas, permite testes em mapas pequenos e é o que viabiliza o modelo "BigBrain": conteúdo novo registra layers sem tocar no núcleo.

3. Percepção

A percepção transforma o mundo em **estímulos com incerteza**. A base pode ser o `UAIPerceptionComponent` da Unreal (`UAISense_Sight`, `UAISense_Hearing`, `UAISense_Damage`, `UAISense_Team`), estendido com modelos próprios onde o padrão é simples demais para um jogo tático.

3.1 Visão

Cone e regiões

- **Central** (cerca de ±35°): taxa de detecção total.
- **Periférica** (até cerca de ±90°): taxa reduzida (por exemplo 25 a 40%).
- Fora do cone: zero, exceto por som, dano ou aviso do esquadrão.

Pontos de amostragem

Um único raycast ao centro do corpo é pouco realista. Implemente `IAISightTargetInterface::CanBeSeenFrom` no pawn alvo e teste **N pontos** (cabeça, tronco, pelve, ombros, pernas). O resultado é uma **fração de exposição** (0 a 1), que modula a velocidade de detecção. Um jogador com só a cabeça à mostra é detectado mais devagar que um totalmente exposto.

Fatores da taxa de detecção

Fator	Descrição	Valor inicial
Distância	Queda quadrática entre `Dnear` e `Dfar`.	Dnear 10 m, Dfar 70 m (ajustar à escala do mapa)
Exposição	Pontos visíveis dividido pelos amostrados.	0 a 1
Movimento	Alvo parado é mais difícil de notar que alvo correndo.	Parado 0.5, andando 1.0, correndo 1.5
Postura	Agachado/deitado reduz silhueta.	Agachado 0.7, deitado 0.45
Luz	Zona escura, lanterna ligada, tiro com clarão.	Escuro 0.5, claro 1.0, lanterna 1.6
Fumaça/vegetação	Volumes que atenuam ou bloqueiam; canal de trace dedicado.	Fumaça densa 0, vegetação 0.4
Periferia	Região do cone onde o alvo está.	Central 1.0, periférico 0.3
Perfil do bot	Multiplicador por dificuldade/personalidade.	0.6 a 1.4

3.2 Acumulador de consciência

A detecção não é binária. Cada ameaça possui um acumulador `Awareness` que sobe quando visível e decai quando não. Isso gera o "quase me viu" que torna furtividade e flanco viáveis.

```
rate        = BaseRate × Dist(d) × Exposure × Motion × Stance × Light × Periphery × Profile.AwarenessMul
Dist(d)     = (1 − saturate((d − Dnear) / (Dfar − Dnear)))²

visível:     Awareness = clamp(Awareness + rate·Δt, 0, 1.5)
não visível: (após grace de ~0.4 s)  Awareness = max(Awareness − Decay·Δt, 0)
```

Estado da ameaça	Condição de entrada	Efeito no comportamento
Desconhecida	Awareness = 0	Rotina normal (patrulha, objetivo).
Suspeita	Awareness ≥ 0.3	Vira a cabeça, para, emite bark "ouvi algo". Ativa Investigar.
Detectada	Awareness ≥ 1.0	Revela posição ao esquadrão, ativa Engage.
Engajada	Detectada e visível ou atirando	Combate pleno, usa cobertura e táticas.
Buscando	Perdeu de vista, Confidence > 0.2	Vai à última posição conhecida e varre a área.
Perdida	Confidence ≤ 0.2	Volta ao objetivo; mantém alerta elevado por um tempo.

3.3 Audição

Eventos sonoros são reportados com `UAISense_Hearing::ReportNoiseEvent(World, Local, Loudness, Instigador, MaxRange, Tag)`. O modelo próprio atenua por distância e por oclusão e devolve uma **posição estimada com erro**.

```
Perceived = Loudness × 1 / (1 + (d_efetiva / d0)²) × Oclusão
d_efetiva = max(distânciaEuclidiana, 0.8 × distânciaPeloNavMesh)    // som não atravessa paredes de graça
Oclusão   = 0.6 ^ (número de superfícies sólidas entre emissor e ouvinte)
erroAngular(°) = Kerr × (1 − Perceived) × (d / 10 m)                 // som fraco e distante = posição imprecisa
```

Evento	Loudness	Alcance máx. (m)	Observação
Tiro sem supressor	1.0	80	Também gera Awareness para toda a área.
Tiro com supressor	0.3	25	
Explosão	1.0	120	Força investigação mesmo de bots em Patrulha.
Passo correndo	0.5	15	Depende do piso (metal, madeira, terra) via Physical Material.

Evento	Loudness	Alcance máx. (m)	Observação
Passo andando	0.2	6	
Passo agachado	0.05	3	Quase inaudível: recompensa jogo furtivo.
Porta/objeto	0.4	12	Interação gera ruído.
Bala passando (whiz)	0.6	15	Revela direção de origem aproximada.
Fala / bark	0.5	20	Bots aliados também ouvem barks inimigos.

3.4 Dano e equipe

- `UAISense_Damage`: ser atingido revela a **direção aproximada** do atirador, mesmo fora do cone. Confidence moderada (0.5), porque o bot sabe de onde veio, não necessariamente onde está.
- `UAISense_Team`: informação de aliados chega com latência e perda de fidelidade (seção 8.2), nunca instantânea e perfeita.

3.5 Interface com as crenças

Todo sensor produz o mesmo tipo de evento: `FBotStimulus { Source, Type, Location, Strength, Time, bFromTeam }`. O componente de percepção **não decide nada**; apenas normaliza e entrega às crenças. Isso permite adicionar sentidos novos (cheiro, câmera, drone) sem tocar no brain.

> **Custo de visão**
> Raycasts múltiplos por par bot/alvo escalam rápido. Faça **early-out** barato antes de traçar (distância, cone, cache de LOS com TTL de 0.1 a 0.2 s), use traces assíncronos quando possível e limite `MaxTracesPerTick` em `UAISense_Sight` (configurável em ini).

4. Modelo de conhecimento (crenças)

O brain decide com base no que o bot **acredita**, não no que é verdade. Esse é o item mais importante para evitar a sensação de IA onisciente.

4.1 Estruturas

```
UENUM() enum class EThreatState : uint8 { Suspected, Detected, Engaged, Searching, Lost };

USTRUCT()
struct FThreatBelief
{
    GENERATED_BODY()
    TWeakObjectPtr<AActor> Actor;
    FVector  LastKnownLocation = FVector::ZeroVector;
    FVector  LastKnownVelocity = FVector::ZeroVector;
    float    Awareness   = 0.f;     // acumulador de percepção (seção 3.2)
    float    Confidence  = 0.f;     // 0..1: certeza sobre ONDE a ameaça está agora
    double   FirstSeenTime = 0.0, LastSeenTime = 0.0, LastHeardTime = 0.0;
    EThreatState State = EThreatState::Suspected;
    bool     bVisibleNow = false;
    bool     bSharedBySquad = false;
};
```

4.2 Regras de atualização

- **Visível:** posição e velocidade reais amostradas, `Confidence = 1`.
- **Não visível:** `Confidence` decai (por exemplo −0.15/s). A posição passa a ser **extrapolada** (`LastKnownLocation + Velocity × t`, limitada a poucos segundos e restrita ao NavMesh).
- **Som:** cria ou atualiza a crença com posição ruidosa e `Confidence` proporcional à intensidade percebida.
- **Dano:** direção do tiro vira crença de baixa confiança.
- **Área de busca:** quando o alvo é perdido, amostrar pontos do NavMesh dentro de um raio que cresce com o tempo, descartando os que o bot já viu vazios. Esses pontos alimentam as queries EQS de busca.

4.3 Memória de longo prazo

- Áreas já varridas e liberadas (evita revisitar).
- Locais onde bots do esquadrão morreram (ponto quente, evitar ou preparar emboscada).
- Rotas por onde o jogador costuma aparecer (influência leve, opcional).

4.4 Regras anti-wallhack

> **Regra de ouro (code review)**
> Nenhuma layer, EQS context ou task pode chamar `UGameplayStatics::GetPlayerPawn`/`GetAllActorsOfClass` para obter a posição do jogador. Todo acesso a inimigos passa por `UBotBeliefComponent`. Crie uma checagem automática (grep em CI ou regra de lint) para garantir isso.

5. Decisão: brains e layers

5.1 Conceito

Um **brain** é uma lista ordenada de **layers**. Cada layer sabe responder duas perguntas: "devo rodar agora?" (`Evaluate`) e "o que faço?" (`Tick`). A cada ciclo de decisão, o arbiter percorre as layers da maior para a menor prioridade e **a primeira que quiser rodar vence**. É a mesma ideia do sistema de layers do jogo base que o BigBrain expõe a mods.

5.2 Contrato de uma layer

```
UENUM() enum class EBotLayerStatus : uint8 { Running, Succeeded, Failed };

UCLASS(Abstract, Blueprintable, EditInlineNew, DefaultToInstanced)
class TACTICALAI_API UBotLayer : public UObject
{
    GENERATED_BODY()
public:
    UPROPERTY(EditDefaultsOnly) FName LayerId;
    UPROPERTY(EditDefaultsOnly) int32 Priority = 0;
    UPROPERTY(EditDefaultsOnly) float MinRunTime = 0.6f;      // anti-flicker
    UPROPERTY(EditDefaultsOnly) bool  bInterruptible = true;

    // 0 = não quer rodar. >0 = quer rodar (valor desempata layers de mesma prioridade)
    virtual float Evaluate(const FBotContext& Ctx) const { return 0.f; }
    virtual void  OnEnter(FBotContext& Ctx) {}
    virtual EBotLayerStatus Tick(FBotContext& Ctx, float DeltaTime) { return EBotLayerStatus::Running; }
    virtual void  OnExit(FBotContext& Ctx, EBotExitReason Reason) {}
};
```

`FBotContext` é um objeto leve (struct) com ponteiros para controller, pawn, crenças, esquadrão, perfil e escritor de intents. É a **única** porta de entrada da layer para o resto do sistema.

5.3 Arbiter

```
void UBotBrainComponent::Decide(double Now)
{
    UBotLayer* Best = nullptr;
    for (UBotLayer* L : SortedLayers)                        // prioridade decrescente
        if (L->Evaluate(Ctx) > 0.f) { Best = L; break; }

    if (Best == Active) return;

    const bool bCanSwitch = !Active
        || (Active->bInterruptible && (Now - ActiveSince) >= Active->MinRunTime)
        || Best->Priority >= EmergencyPriority;              // emergências furam o MinRunTime

    if (!bCanSwitch) return;
    if (Active) Active->OnExit(Ctx, EBotExitReason::Interrupted);
    DecisionTrace.Push({Now, Active, Best, Ctx.DebugReason});
    Active = Best; ActiveSince = Now; Active->OnEnter(Ctx);
}
```

5.4 Catálogo de layers

Prior.	Layer	Ativa quando	Ação principal	Interrompível
100	Morto / ragdoll	HP ≤ 0	Libera cobertura, tokens e papel no esquadrão.	Não
95	Atordoado / cegado	Flashbang, stun	Cobre o rosto, tiro cego, sem percepção.	Não
90	Travessia	Nav link em uso (vault, escada)	Executa a animação até o fim.	Não
80	Sobreviver	HP abaixo do limiar do perfil	Recuar para cobertura segura, curar.	Sim
75	Evitar granada	Granada no raio de perigo	Fugir do raio, jogar-se ao chão.	Não
70	Engage (combate)	Ameaça Detectada/Engajada	Ciclo de combate (seção 7).	Sim
60	Sob supressão	Supressão acima do limiar	Abaixar, ficar em cobertura, tiro cego.	Sim
55	Ordem do esquadrão	Ordem pendente do `ASquad`	Executar papel (flanquear, suprimir, avançar).	Sim
50	Investigar	Ameaça Suspeita ou som	Ir ao ponto, olhar, decidir.	Sim
45	Buscar	Ameaça Buscando	Varrer pontos de busca (EQS).	Sim
40	Objetivo	Missão atribuída	Defender, ocupar, escoltar, assaltar.	Sim
30	Patrulha	Sem objetivo ativo	Rota ou pontos de interesse.	Sim
10	Ocioso	Sempre (fallback)	Idle com variações.	Sim

Dica de ordenação
Prioridade expressa **urgência**, não "importância". Sobreviver (80) fica acima de Engage (70) porque um bot com 10% de vida precisa decidir fugir antes de pensar em atirar. Evite criar mais de uma dúzia de camadas: se precisa, a lógica provavelmente pertence a uma sub-máquina dentro de uma layer existente.

5.5 Dentro de uma layer

A layer responde **"o que eu quero"**; a sub-lógica responde **"como faço"**. Dentro de Engage, por exemplo, usa-se uma StateTree ou Behavior Tree com os estados do ciclo de combate. Comparação das técnicas:

Técnica	Pontos fortes	Limites	Uso recomendado aqui
Layers por prioridade	Simples, previsível, fácil de depurar e estender.	Não planeja sequências longas.	**Seleção de alto nível** (arbiter).

Técnica	Pontos fortes	Limites	Uso recomendado aqui
Behavior Tree	Composição visual, ampla documentação.	Reatividade depende de bom uso de decorators/abort.	Sub-lógica de Patrulha, Investigar.
StateTree	Estados e transições explícitos, bom desempenho, integra com Mass/Smart Objects.	Ecossistema mais novo; confira recursos na sua versão.	**Sub-lógica de Engage** e ciclos de cobertura.
Utility AI	Comportamento fluido por pontuação.	Ajuste de curvas exige disciplina.	Escolha de posição, alvo e tática do esquadrão.
GOAP	Planeja sequências, emergente.	Difícil de depurar e de prever custo.	Opcional: objetivos de missão complexos.
HTN	Planos hierárquicos controláveis.	Autoria cara.	Planner do esquadrão, se necessário.

5.6 Extensibilidade (o equivalente ao BigBrain)

```
USTRUCT()
struct FBotLayerEntry
{
    GENERATED_BODY()
    UPROPERTY(EditAnywhere, Instanced) TObjectPtr<UBotLayer> Layer;
    UPROPERTY(EditAnywhere) int32 PriorityOverride = INDEX_NONE;
    UPROPERTY(EditAnywhere) FGameplayTagQuery Requires;      // ex.: Map.Factory AND Role.Marksman
};

UCLASS()
class UBotBrainDefinition : public UPrimaryDataAsset
{
    GENERATED_BODY()
public:
    UPROPERTY(EditAnywhere) FGameplayTag BrainTag;           // Brain.Rifleman, Brain.Boss.Heavy ...
    UPROPERTY(EditAnywhere) TArray<FBotLayerEntry> Layers;
};

// API estilo BigBrain: plugins registram/removem layers sem editar o núcleo
UBotBrainRegistry::Get().AddLayer(TAG_Brain_Rifleman, ULayer_Ambush::StaticClass(), 65);
UBotBrainRegistry::Get().RemoveLayer(TAG_Brain_Rifleman, TAG_Layer_Patrol);
```

As chamadas de registro devem ocorrer ao ativar um **Game Feature Plugin** (por uma `UGameFeatureAction` própria) ou no startup do módulo. Ao criar um bot, o brain é montado juntando a definição base e as entradas registradas cujo `Requires` casa com as tags do bot (mapa, facção, papel).

5.7 Estabilidade (anti-flicker)

- **Histerese:** limiares de entrada e saída diferentes (entra em Sobreviver com HP < 30%, sai com HP > 45%).
- **Tempo mínimo** de execução por layer (`MinRunTime`) e **cooldowns** para ações caras.
- **Commitment:** ações em andamento (arremessar granada, escalar) bloqueiam troca (`bInterruptible = false`).
- **Jitter** no intervalo de decisão para evitar picos sincronizados.
- **Trace de decisão:** buffer circular com as últimas ~32 trocas e o motivo (seção 13).

6. Raciocínio tático

6.1 Dados de cobertura

```
USTRUCT()
struct FCoverPoint
{
    GENERATED_BODY()
    FVector  Location;           // onde o bot fica (atrás da cobertura)
    FVector  Normal;             // direção "frontal" protegida
    ECoverHeight Height;         // Low (agachado) | High (em pé)
    uint8    PeekMask;           // Esquerda | Direita | Topo
    float    ProtectionArc;      // abertura (graus) em que protege
    int32    Capacity = 1;
    // runtime
    TWeakObjectPtr<AActor> ClaimedBy; double ClaimExpiry = 0.0;
};
```

Geração: prefira um **bake offline** (ferramenta de editor, PCG da UE ou geração procedural externa) que analisa a geometria, valida altura, largura, rota de entrada/saída e salva em um DataAsset por nível. Para conteúdo dinâmico, um **fallback em runtime** via EQS (grade + traces) cobre o que o bake não vê. Cada ponto guarda atributos estáticos; reservas e ocupação são estado de runtime no `UCoverSubsystem`.

6.2 Pontuação de posições

Primeiro **filtros duros** (alcançável, não reservado, não exposto à ameaça pela frente, fora de raios de perigo). Depois **pontuação ponderada**: `Score = Σ (peso × nota)` de cada componente. Os pesos vêm do perfil, e é isso que diferencia um bot agressivo de um cauteloso.

Componente	Mede	Peso típico
Proteção	Quanto a cobertura bloqueia a linha das ameaças conhecidas (ângulo e altura).	+3.0
Campo de tiro	Ao peekar, vê a ameaça? Distância na faixa ideal da arma?	+2.0
Proximidade	Distância de caminho até o ponto (menos é melhor).	+1.0
Ângulo de flanco	Ângulo em relação à ameaça; flanquear pontua mais.	+0.0 a +2.0 (perfil)
Rota de fuga	Existe um segundo ponto seguro atrás?	+1.0
Aglomeração	Quantos aliados estão perto (vulnerável a granada).	−1.5
Recência	Usou este ponto há pouco (evita ficar previsível).	−1.0
Exposição no trajeto	Custo de ficar exposto no caminho até lá.	−2.0

Ordem de custo
Ordene os testes do mais barato para o mais caro: distância e filtros de reserva primeiro, traces de linha de visão por último e só nos N melhores candidatos. No EQS, isso significa colocar testes baratos antes e usar a pontuação parcial para descartar.

6.3 Reservas

- Ao escolher um ponto, o bot faz **claim** com TTL (por exemplo 8 s) e o renova enquanto o ocupa.
- Ao sair, morrer ou perder o ponto, libera. Sem isso, pontos ficam "fantasma".
- **Roubo de reserva** apenas por bot de papel/prioridade maior (por exemplo, líder) e com aviso ao dono para reescolher.

6.4 Catálogo de queries EQS

Query	Gerador	Testes principais
FindCover	Pontos do `UCoverSubsystem` em raio	Reserva, proteção vs. ameaça, distância de caminho, campo de tiro.
FindFlankPosition	Pontos em arco lateral à ameaça	Ângulo mínimo, rota sem LOS à ameaça, distância ao alvo.
FindAdvancePosition	Pontos à frente na direção da ameaça	Cobertura intermediária, exposição no trajeto.
FindSearchPoints	Pontos do NavMesh na área de busca	Não visitado, visibilidade da área, proximidade.
FindRetreat	Pontos atrás do bot	Longe da ameaça, com cobertura, perto de aliados.
FindGrenadeThrow	Pontos com alcance ao alvo	Trajetória livre, distância de segurança, sem aliados no raio.

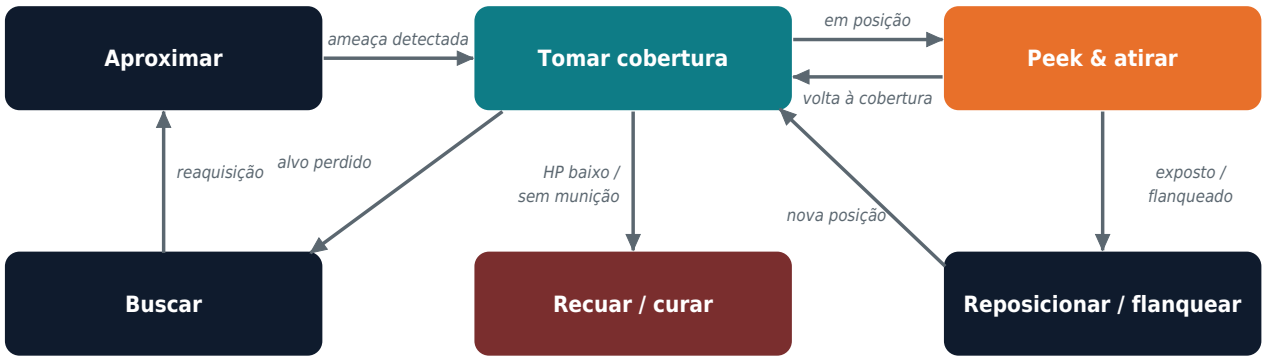
6.5 Movimento sob fogo

O caminho mais curto raramente é o mais inteligente. Use um **filtro de navegação** (`UNavigationQueryFilter`) com áreas de custo (`UNavArea`) e modificadores dinâmicos (`UNavModifierComponent`) para marcar regiões expostas à ameaça conhecida. Assim, sob fogo, o bot prefere contornar por trás de obstáculos mesmo que o trajeto seja mais longo.

6.6 Granadas

- **Quando:** ameaça conhecida em cobertura há mais de N segundos, sem LOS direto, e o bot possui granada.
- **Onde:** posição de arremesso via query EQS; trajetória via `UGameplayStatics::SuggestProjectileVelocity`.
- **Segurança:** sem aliados no raio, distância mínima do próprio bot, checagem de teto/obstáculos.
- **Controle:** requer token de granada do esquadrão e cooldown por bot (seção 8.5).

7. Ciclo de combate



Qualquer estado pode ser interrompido por layers de maior prioridade (Sobreviver, Granada, Ordem de esquadrão). A camada Engage orquestra este ciclo; cada estado é uma sub-máquina (StateTree/BT) com saídas bem definidas.

Figura 2. Ciclo de combate dentro da layer Engage.

Estado	Entrada	Comportamento	Saída
Aproximar	Ameaça detectada e distância fora da faixa da arma.	Avançar por cobertura intermediária, atirar se exposto.	Chegou à posição ou foi atingido.

Estado	Entrada	Comportamento	Saída
Tomar cobertura	Sob fogo ou posição escolhida.	Mover e agachar/colar na cobertura, reservar ponto.	Em posição (estável por 0.3 s).
Peek & atirar	Em cobertura com alvo conhecido.	Pausa, levanta/inclina, rajada, volta. Duração e pausa vêm do perfil.	Rajada completa, munição baixa, dano recebido.
Reposicionar / flanquear	Cobertura comprometida ou ordem de flanco.	Mover para nova posição via EQS, com supressão do esquadrão.	Nova posição alcançada.
Recuar / curar	HP baixo, sem munição ou moral baixa.	Recuar para cobertura segura, curar/recarregar fora de LOS.	Condição recuperada ou ameaça sumiu.
Buscar	Alvo perdido, Confidence > 0.2.	Ir à última posição e varrer pontos EQS.	Reaquisição (volta a Engage) ou desistência.

7.1 Moral (lutar ou fugir)

Uma pontuação de moral simples já produz comportamento crível: bots abandonam a luta quando estão sozinhos, feridos e sob pressão, e ficam ousados com aliados por perto.

```
Morale = clamp( Base                          // do perfil (Disciplina)
              + 0.30 × aliadosPróximos/3
              − 0.25 × ameaçasConhecidas/3
              − 0.40 × Supressão               // 0..1
              − 0.30 × (1 − HP/HPmax)
              − 0.15 × (munição < 25%) , 0, 1 )
// Morale < limiar → layer Sobreviver / Recuar.   Histerese: voltar só com Morale > limiar + 0.15
```

7.2 Supressão

Tiros que passam perto acumulam `Supressão` (0 a 1), que decai com o tempo. Efeitos: aumenta o erro de mira, reduz a chance de peek, aumenta a de ficar baixo e alimenta a moral. Bots também aplicam supressão no jogador: ruído e pressão para abrir espaço ao flanco.

8. Esquadrão

Bots isolados parecem espertos; **bots coordenados parecem inteligentes**. O esquadrão é um objeto de primeira classe, com conhecimento, planos e recursos compartilhados.

8.1 Estrutura

- `ASquad` (um por grupo): lista de membros, líder, papéis, plano atual, tokens, conhecimento compartilhado.
- `USquadSubsystem`: registro, criação e destruição de esquadrões, consultas globais (por exemplo, quantos bots atacam agora).
- O líder é eleito (maior prioridade/patente); se morrer, o próximo assume, com um bark e breve reavaliação.

8.2 Conhecimento compartilhado

Para evitar "mente coletiva", a informação **viaja**: quando um membro detecta, os demais recebem após 0.5 a 1.5 s (rádio/grito), com `Confidence` reduzida (por exemplo ×0.7) e ruído na posição. Se o emissor morre antes de transmitir, a informação se perde.

8.3 Planner do esquadrão

A cada 0.5 a 1.0 s o planner reavalia a situação e escolhe uma **tática** por pontuação (Utility) ou um pequeno HTN:

Tática	Quando pontua alto	Atribuições
Segurar posição	Superioridade defensiva, objetivo a defender.	Todos em cobertura, 1 sentinela.
Avançar em lances	Ameaça conhecida, boa cobertura intermediária.	Um grupo suprime, outro avança, alterna.
Suprimir + flanquear	Ameaça fixa em cobertura, rota lateral livre.	Suppressor + Flanker (token de flanco).
Cercar / buscar	Ameaça perdida.	Dividir áreas de busca entre membros.
Recuar	Baixas altas, moral baixa.	Fogo de cobertura enquanto recuam.
Invadir (breach)	Cômodo fechado, ameaça conhecida lá.	Posições na porta, granada/flash, entrada.

8.4 Papéis

Papel	Função	Critérios de atribuição
Ponta (point)	Entra primeiro em áreas desconhecidas.	Maior HP/armadura, perfil agressivo.
Supressor	Mantém a ameaça agachada com fogo.	Arma automática, bastante munição.
Flanqueador	Contorna a ameaça.	Perfil ousado, caminho livre disponível.
Suporte	Cobre, joga granadas, ocupa pontos-chave.	Posição estável, granada disponível.
Retaguarda	Observa as costas do esquadrão.	Bot mais distante da ameaça.

8.5 Tokens

Tokens limitam quantos bots podem executar uma ação "cara em atenção do jogador" ao mesmo tempo. É o mecanismo que evita 5 bots atirando em sequência no mesmo instante e garante lutas justas e legíveis.

Token	Máx. simultâneo	Cooldown	Finalidade
Ataque	2 a 4 (por dificuldade)	–	Quem pode atirar com intenção de acertar. Os demais ficam em cobertura/supressão passiva.
Flanco	1	10 a 20 s	Um flanco por vez.
Granada	1	8 a 15 s	Evita enxurrada de granadas.
Fala (bark)	1	1.5 s	Evita sobreposição de vozes.

```
bool USquad::TryAcquireToken(ETokenType Type, AActor* Who)
{
    FSquadToken& T = Tokens[Type];
    const double Now = GetWorld()->GetTimeSeconds();
    if (T.Holders.Contains(Who)) return true;
    if (Now < T.CooldownUntil || T.Holders.Num() >= T.Max) return false;

    // anti-starvation: com fila, só o membro de maior "idade de espera" leva
    T.WaitSince.FindOrAdd(Who, Now);
    if (const AActor* Oldest = T.OldestWaiter(); Oldest && Oldest != Who) return false;

    T.Holders.Add(Who); T.WaitSince.Remove(Who);
    return true;
}
```

8.6 Comunicação (barks)

Barks têm duas funções: **informar o jogador** (telegraphing) e **dar vida ao esquadrão**. Gerencie por fila de prioridade, categoria, cooldown por categoria e por bot, e sincronia com animação/lábios.

Categoria	Exemplos de situação	Prioridade
Contato	Detectou o jogador, viu a posição.	Alta
Tática	Flanqueando, avançando, cobrindo, granada!	Média
Estado	Recarregando, ferido, sem munição.	Média
Baixa	Aliado morto, líder caiu.	Alta
Ambiente	Ouvi algo, área limpa.	Baixa

8.7 Espaçamento e fogo amigo

- Distância mínima entre aliados (2 a 3 m) para não agruparem (vulneráveis a granada).
- Checagem de **linha de fogo**: o bot não atira se houver aliado na trajetória.
- Cobertura compartilhada só quando `Capacity` permite.

9. Execução e humanização

9.1 Movimento

- NavMesh Recast; `UCrowdFollowingComponent` (Detour) ou RVO para evitar colisões entre bots.
- **Nav links** inteligentes para vault, escalar, pular; a layer Travessia bloqueia interrupção.
- **Perfis de velocidade** (andar, correr, agachado) ligados ao ruído emitido (seção 3.3).
- **Cantos:** técnica "slice the pie" (abrir ângulo gradualmente) em vez de virar a esquina de frente.
- Detecção de **travamento** (stuck): sem progresso por N s dispara reescolha de rota e log para debug.

9.2 Mira falível

A mira é o que mais define "dificuldade justa". Em vez de multiplicar dano, **modele o erro**: reação, aquisição, velocidade de rotação, dispersão e tempo no alvo.

Componente	Modelo
Reação	Tempo entre ver e começar a mirar, amostrado de uma distribuição (por exemplo log-normal) por perfil.
Aquisição	Rotação com mola amortecida e velocidade angular máxima; não "snapa" no alvo.
Ponto de mira	Centro de massa por padrão; cabeça só para perfis hábeis e com exposição.
Erro	Cone de dispersão que cresce com distância, movimento próprio, supressão e dano recente.
Tempo no alvo	Erro diminui conforme o bot mantém o alvo; zera ao trocar de alvo.
Rajada	Tamanho e pausa dependem da arma e do perfil; controle de recuo imperfeito.

```
FVector UAimModel::ComputeAimDirection(const FAimInput& In) const
{
    const float Skill = In.Profile->Accuracy;                    // 0..1
    float ErrDeg = In.Profile->BaseAimErrorDeg * (1.f - 0.7f * Skill);
    ErrDeg *= 1.f + KDist * In.DistanceM + KMove * In.SelfSpeed
                  + KSupp * In.Suppression + KDmg * In.RecentDamage;
    ErrDeg /= FMath::Lerp(1.f, 2.5f, In.TimeOnTargetNorm);          // melhora ao "travar" no alvo
    return FMath::VRandCone(In.IdealDir, FMath::DegreesToRadians(ErrDeg));
}
```

9.3 Animação

- **Aim offset** para mirar sem girar o corpo todo; camada de upper body para tiro/recarga.
- Peek/lean por animação ou deslocamento procedural, sincronizado ao ciclo de combate.
- **Anim notifies** disparam o tiro para que animação e dano coincidam.
- Reações a dano e a supressão com variação para evitar repetição.

9.4 Justiça e legibilidade

- **Primeiro tiro com atraso** (grace) após notar o jogador, o que dá chance de reação.
- **Quase-acertos** antes dos acertos: tiros próximos que revelam a direção, em vez de hitscan perfeito.
- **Telegraphing:** lanterna, laser, bark e gesto antes de uma investida ou granada.
- **Rampa de precisão:** a pontaria melhora ao longo dos primeiros segundos de combate.
- **Teto de atacantes simultâneos** via tokens, escalado por dificuldade.
- Evitar dano oculto ou visão por parede: o jogador deve conseguir explicar a derrota.

9.5 Imperfeições deliberadas

- Hesitação e mudança de ideia com atrasos plausíveis.
- Pequenos erros: recarregar fora de cobertura, esquecer de checar um canto.
- Posição do jogador "escondido" imprecisa: busca em área, não em ponto exato.

10. Perfis, personalidades e dificuldade

10.1 UBotProfile

```
UCLASS()
class UBotProfile : public UPrimaryDataAsset
{
    GENERATED_BODY()
public:
    // Cognição
    UPROPERTY(EditAnywhere) FFloatRange ReactionTime;        // s
    UPROPERTY(EditAnywhere) float AwarenessMul = 1.f;
    UPROPERTY(EditAnywhere) float SightRangeM  = 70.f;
    // Combate
    UPROPERTY(EditAnywhere) float Accuracy = 0.5f;           // 0..1
    UPROPERTY(EditAnywhere) float BaseAimErrorDeg = 3.f;
    UPROPERTY(EditAnywhere) float BurstLength = 4.f;
    // Personalidade
    UPROPERTY(EditAnywhere) float Aggression = 0.5f;         // 0..1
    UPROPERTY(EditAnywhere) float Discipline = 0.5f;         // moral base, obediência
    UPROPERTY(EditAnywhere) float RetreatHP = 0.3f;
    // Táticos: pesos de cobertura e flanco, uso de granada, ...
    UPROPERTY(EditAnywhere) FTacticalWeights Weights;
    // Ajustes de brain
    UPROPERTY(EditAnywhere) TArray<FLayerPriorityOverride> LayerOverrides;
};
```

10.2 Faixas de dificuldade (ponto de partida)

Parâmetro	Recruta	Regular	Veterano	Elite	Chefe
Reação (s)	0.9 a 1.4	0.6 a 0.9	0.4 a 0.6	0.25 a 0.4	0.2 a 0.3
Erro de mira base (°)	6.0	4.0	2.5	1.5	1.0
Multiplicador de percepção	0.6	0.8	1.0	1.2	1.4
Alcance de visão (m)	40	55	70	90	100
Atacantes simultâneos (token)	1	2	2	3	4
Usa granadas	Não	Raro	Sim	Sim	Sim
Flanqueia	Não	Raro	Sim	Sim	Sim
Limiar de recuo (HP)	50%	40%	30%	25%	10%

10.3 Personalidades

Personalidade	Efeito	Ajustes típicos
Agressivo / rusher	Pressiona, aceita risco, ignora cobertura ideal.	Aggression alta; peso de "avanço" alto; RetreatHP baixo.
Cauteloso	Prefere cobertura, flanco lento, recua cedo.	Peso de proteção alto; RetreatHP alto; Discipline média.
Disciplinado	Segue ordens e formação, bom trabalho em equipe.	Discipline alta; peso de ordem do esquadrão alto.
Atirador (marksman)	Longa distância, paciente, mira precisa.	Accuracy alta; faixa ideal longa; reação lenta, erro baixo.
Imprevisível	Varia táticas e ritmo.	Pesos randomizados por encontro, dentro de limites.

10.4 Dificuldade dinâmica (opcional)

Com telemetria (taxa de mortes do jogador, tempo por encontro), é possível ajustar suavemente tokens e reação dentro de limites. Mantenha **transparente e limitado**: o jogador não deve perceber o jogo "aliviando" de forma óbvia, e modos competitivos devem desativá-la.

11. Multiplayer e rede

- **Autoridade no servidor:** `AAIController` existe só no servidor. Percepção, decisão e disparo rodam lá.
- **Replicação:** replique apenas estado do pawn (postura, rotação de mira via `GetBaseAimRotation`, flags de animação), não o estado interno da IA.
- **Barks e efeitos:** multicast RPC não confiável ou evento replicado; o áudio toca localmente.
- **Hit registration:** tiros de bots usam posições do servidor; verifique se a experiência do jogador com latência permanece justa (por exemplo, pequena tolerância ao acertar jogadores com ping alto).
- **Custo no servidor:** orçamentos por bot (seção 12) valem em dedicated server; considere reduzir tick de bots distantes dos jogadores.
- **Determinismo:** não é necessário, mas **seed registrada** por encontro facilita reproduzir bugs.

12. Performance

12.1 Orçamentos

Defina os orçamentos **antes** de escalar o número de bots e meça sempre em hardware-alvo. Valores iniciais sugeridos para um jogo com dezenas de bots ativos:

Recurso	Orçamento inicial	Mecanismo
Decisão do brain	0.1 a 0.3 s por bot, escalonado	Jitter e fila circular.
Traces de visão	Limite por tick	`MaxTracesPerTick` e early-outs.
EQS	Tempo máximo por frame	`MaxAllowedTestingTime` no `UEnvQueryManager`; cache de resultados.
Custo por bot	Meta amortizada abaixo de ~0.1 a 0.15 ms	Medir com Unreal Insights e ajustar.
Alocações em runtime	Zero por tick	Structs, pools, evitar criar UObjects por decisão.

12.2 AI LOD

Nível	Critério	Comportamento
Pleno	Em combate ou perto/visível ao jogador	Percepção e decisão completas, animação total.
Reduzido	Distante, fora de vista	Decisão a 0.5 s, percepção simplificada, animação barata.
Dormente	Muito distante/irrelevante	Patrulha simplificada, ou removido e reativado por evento.

A classificação pode usar `USignificanceManager` ou lógica própria baseada em distância, visibilidade e estado de combate.

12.3 Técnicas

- **Escalonamento (stagger):** distribua decisões e queries por frames diferentes.
- **Cache:** LOS, cobertura e resultados de EQS com TTL curto.
- **Traces assíncronos** (`AsyncLineTraceByChannel`) para percepção.
- **Early-out:** descartar por distância, cone e estado antes de qualquer trace.
- **Profiling:** escopos `TRACE_CPUPROFILER_EVENT_SCOPE` por módulo e `DECLARE_CYCLE_STAT` para estatísticas.

13. Debug e ferramentas

Ferramenta	Uso
Gameplay Debugger (tecla apóstrofo)	Categoria própria mostrando layer ativa, crenças com confiança, cobertura reservada, tokens e papel do bot.
Visual Logger	`UE_VLOG_*` para registrar decisões, linhas e esferas ao longo do tempo, com scrubbing.
Decision trace	Buffer circular por bot: tempo, layer anterior, layer nova, **motivo** e pontuações. Responde "por que ele fez isso?".
Visualizadores de editor	Cobertura (normais, capacidade, peek), pontos de patrulha, zonas táticas.
EQS Testing Pawn	`AEQSTestingPawn` para ajustar queries diretamente no mapa.
Comandos de console	`ai.bot.debug <id>`, `ai.bot.freeze`, `ai.bot.forcelayer <layer>`, `ai.bot.step`.
Gravação de cenários	Registrar estímulos e seeds para reproduzir um encontro específico.

14. Testes e qualidade

14.1 Camadas de teste

- **Unitários** (`FAutomationSpec`): matemática de crenças, arbiter, tokens, pontuação. Lógica pura em C++, sem mundo.
- **Funcionais** (`AFunctionalTest`) em arenas pequenas, com cenários repetíveis.
- **Soak:** simulações bot contra bot, headless, por horas, para pegar travamentos, vazamentos e picos.
- **Playtests** com telemetria (mapas de calor de mortes, tempo por encontro).

14.2 Cenários de arena

Cenário	Resultado esperado
Jogador aparece no flanco	Bot detecta com atraso, vai à cobertura, avisa o esquadrão.
Jogador some atrás de parede	Bot busca na última posição e varre, sem "saber" onde está.
Tiro com e sem supressor	Raio e confiança de detecção condizem com a tabela de som.
Cobertura única para dois bots	Um reserva; o outro escolhe alternativa.
Esquadrão de 4 sob fogo	No máximo N atacam por vez; os demais suprimem ou reposicionam.
Líder morre	Novo líder eleito, bark e reavaliação do plano.
HP baixo e sem aliados	Recua e cura fora de LOS; volta com histerese.

14.3 Métricas

Métrica	Para que serve
Tempo até o primeiro tiro (TTFS)	Calibra reação e justiça.
Percentual de uso de cobertura	Verifica se o ciclo tático está funcionando.
Sucesso de flanco	Mede eficácia do planner.
Taxa de starvation de tokens	Detecta bots que nunca atacam.
Taxa de stuck / troca de layer por segundo	Detecta travamento e flicker.
Latência p95 de decisão e custo por bot	Vigia performance.

15. Roadmap de implementação

Ordem sugerida de construção, em fases incrementais. Cada fase termina com algo **jogável e mensurável**. As estimativas são indicativas para 1 a 2 engenheiros de gameplay e variam com experiência e escopo.

Fase	Entregas	Critério de pronto	Estimativa
F0 Fundações	Plugin, controller, `FBotContext`, intents, debug básico, arena de testes.	Bot anda e atira em alvo fixo; HUD de debug.	1 a 2 sem.
F1 Percepção + crenças	Visão com pontos, acumulador, som, modelo de crenças.	Bot "vê/ouve" com incerteza; crenças visíveis no debugger.	2 a 3 sem.
F2 Brain + layers	Arbiter, 4 a 5 layers (Ocioso, Patrulha, Investigar, Engage simples, Sobreviver).	Troca de layers estável, sem flicker.	2 a 3 sem.
F3 Cobertura + EQS	Bake de cobertura, subsistema, queries, filtro de navegação.	Bot usa cobertura e reserva sem conflito.	3 a 4 sem.

Fase	Entregas	Critério de pronto	Estimativa
F4 Combate completo	Ciclo de combate, mira falível, supressão, moral, granadas.	Combate crível 1 contra 1.	3 a 4 sem.
F5 Esquadrão	Subsistema, conhecimento compartilhado, papéis, tokens, planner, barks.	Esquadrão flanqueia e suprime de forma coordenada.	3 a 4 sem.
F6 Perfis e dificuldade	DataAssets, tiers, personalidades, registro de layers (API estilo BigBrain).	Dificuldades distintas e perceptíveis.	2 a 3 sem.
F7 Performance, rede e polish	AI LOD, orçamentos, replicação, soak tests, ajustes finais.	Metas de custo atingidas; estável em soak.	3+ sem.

16. Riscos e armadilhas comuns

Risco	Sintoma	Mitigação
Bot "perfeito"	Acerta tudo, vê através de objetos, reage instantaneamente.	Crenças + mira falível + atraso de reação; regra anti-wallhack.
Flicker de layers	Troca de comportamento a cada tick.	Histerese, `MinRunTime`, cooldowns, trace de decisão.
Oscilação de cobertura	Bot corre entre dois pontos.	Reserva com TTL, penalidade de recência, compromisso mínimo.
Starvation de tokens	Alguns bots nunca atacam.	Fila por idade de espera, rotação.
Esquadrão previsível	Mesma tática sempre.	Utility com ruído, penalidade de repetição, variação por perfil.
NavMesh com buracos	Bot trava, não acha rota.	Validação de nav no CI, detecção de stuck, nav links.
Picos de EQS/visão	Engasgos quando muitos bots engajam.	Orçamento por frame, cache, escalonamento, AI LOD.
Ajuste infernal	Parâmetros demais, ninguém sabe o efeito.	Perfis versionados, curvas nomeadas, métricas automáticas, mudar uma coisa por vez.
Acoplamento	Layers dependem umas das outras.	Comunicação só por crenças, intents e eventos.

17. Apêndices

A. Checklist de implementação / code review

- Nenhuma layer acessa o mundo diretamente; só `FBotContext`, crenças e intents.
- Toda troca de layer registra um motivo no decision trace.
- Todo estado que depende de tempo usa histerese ou `MinRunTime`.
- Reservas (cobertura, tokens) têm TTL e liberam em `OnExit` e na morte.
- Nenhuma alocação por tick no caminho quente.
- Parâmetros vêm de DataAssets, sem números mágicos no código.
- Sensores só emitem estímulos; só o brain decide.
- Há cenário de arena e teste automatizado para cada comportamento novo.

B. Glossário

Termo	Significado
Brain	Conjunto ordenado de layers de um tipo de bot.
Layer	Unidade de decisão com condição de ativação e prioridade.
Intent	Pedido de ação escrito pela layer (mover, olhar, atirar) e consumido por executores.
Crença (belief)	Informação do bot sobre uma ameaça, com confiança e idade.
Awareness	Acumulador de consciência sobre uma ameaça (0 a 1.5).
Token	Recurso limitado do esquadrão que autoriza uma ação (atacar, flanquear, granada).
EQS	Environment Query System: consulta espacial por geração e teste de pontos.
Telegraphing	Sinalizar a intenção do bot para que o jogador possa reagir.
AI LOD	Redução de custo de IA conforme relevância para o jogador.
Histerese	Limiares diferentes para entrar e sair de um estado, evitando oscilação.

C. Referências

- J. Orkin, **"Three States and a Plan: The A.I. of F.E.A.R."**, GDC 2006 (GOAP).
- D. Isla, **"Handling Complexity in the Halo 2 AI"**, GDC 2005 (Behavior Trees).
- R. Straatman, A. Beij, W. van der Sterren, **"Killzone's AI: Dynamic Procedural Combat Tactics"**, GDC 2005 (posições táticas).
- **Game AI Pro** (série de livros, capítulos gratuitos em gameaipro.com): percepção, cobertura, esquadrões, utility AI.
- D. Mark, **Behavioral Mathematics for Game AI** (Utility AI e curvas de resposta).
- Documentação da Unreal Engine: AI Perception, Environment Query System, StateTree, Gameplay Debugger, Visual Logger, Smart Objects, Game Features.
- Repositório **SPT-BigBrain** (DrakiaXYZ): referência de arquitetura de brains/layers extensíveis em jogo existente.